コンパイラ演習課題 (3)(解答例)

[1] 生成規則に以下のような番号をつけ，First_1，Follow_1 は以下のようになる．

```
生成規則:
  [1] S -> '(' X
  [2] S -> E ']'
  [3] S -> F ')'
  [4] X -> E ')'
  [5] X -> F ']'
  [6] E -> A
  [7] F -> A
  [8] A -> ε
```

```
FIRST(1):
  FIRST(S) = { '(', ']', ')' }
  FIRST(X) = { ']', ')' }
  FIRST(E) = { ε }
  FIRST(F) = { ε }
  FIRST(A) = { ε }
```

```
FOLLOW(1):
  FOLLOW(S) = { $ }
  FOLLOW(X) = { $ }
  FOLLOW(E) = { ']', ')' }
  FOLLOW(F) = { ']', ')' }
  FOLLOW(A) = { ']', ')' }
```

LL(1) の下向き構文解析表および LALR(1) 構文解析表は以下のように構成される．

	'('	']'	')'	#
S	[1]	[2]	[3]	·
X	·	[5]	[4]	·
E	·	[6]	[6]	·
F	·	[7]	[7]	·
A	·	[8]	[8]	·

状態	'('	']'	')'	#	S	X	E	F	A
0	s1	r8	r8	·	5	·	3	4	2
1	·	r8	r8	·	·	8	6	7	2
2	·	r6 / r7	r7 / r6	·	·	·	·	·	·
3	·	s9	·	·	·	·	·	·	·
4	·	·	s10	·	·	·	·	·	·
5	·	·	·	acc	·	·	·	·	·
6	·	·	s11	·	·	·	·	·	·
7	·	s12	·	·	·	·	·	·	·
8	·	·	·	r1	·	·	·	·	·
9	·	·	·	r2	·	·	·	·	·
10	·	·	·	r3	·	·	·	·	·
11	·	·	·	r4	·	·	·	·	·
12	·	·	·	r5	·	·	·	·	·

r6/r7 で A を E に還元すればよいか F に還元すればよいかを判断できないので，LALR(1) ではない．これに対して下向き構文解析は，A の Follow で規則を判断できる．

[2] (a) LALR(1) 構文解析表は以下のようになる．

```
生成規則:
  [1] S -> E
  [2] E -> 'while' E 'do' E
  [3] E -> ID ':=' E
  [4] E -> E '+' E
  [5] E -> ID
```

状態	ID	'while'	'do'	':='	'+'	#	S	E
0	s3	s1	·	·	·	·	4	2
1	s3	s1	·	·	·	·	·	5
2	·	·	·	·	s6	r1	·	·
3	·	·	r5	s7	r5	r5	·	·
4	·	·	·	·	·	acc	·	·
5	·	·	s8	·	s6	·	·	·
6	s3	s1	·	·	·	·	·	9
7	s3	s1	·	·	·	·	·	10
8	s3	s1	·	·	·	·	·	11
9	·	·	r4	·	s6 / r4	r4	·	·
10	·	·	r3	·	s6 / r3	r3	·	·
11	·	·	r2	·	s6 / r2	r2	·	·

(b) 構文解析表で衝突が発生しているのは，+ が do の範囲内にあるか，範囲外にあるかということがこの文法からは判断できないためである．例えば，while ID do ID=E + E の解釈を (while ID do ID=E) + E とするか，while ID do (ID=E + E) とするか，の違いである．後者の方が解釈としては自然であるので，衝突が起こっているエントリーでは，シフトを優先して do の範囲を大きくする方が自然である．

[3] 講義ページのチェックツールにおいて，文法を記述すると以下のようになる．

```
%start S
%%
S : X
  ;
X : M 'a'
  | 'b' M 'c'
  | 'd' 'c'
  | 'b' 'd' 'a'
  ;
M : 'd'
  ;
%%
```

上記のファイルを ex261016g3.y とする.

!python3 slr1.py ex261016g3.y を実行する.

```
開始記号: S

生成規則:
  [1] S -> X
  [2] X -> M 'a'
  [3] X -> 'b' M 'c'
  [4] X -> 'd' 'c'
  [5] X -> 'b' 'd' 'a'
  [6] M -> 'd'

FIRST:
  FIRST(S) = { 'b', 'd' }
  FIRST(X) = { 'b', 'd' }
  FIRST(M) = { 'd' }

FOLLOW:
  FOLLOW(S) = { # }
  FOLLOW(X) = { # }
  FOLLOW(M) = { 'a', 'c' }

SLR(1) 構文解析表:
ACTION列: sN = Nへ shift、rN = 規則Nで reduce、acc = accept
状態 | 'a'       | 'b' | 'c'      | 'd' | #   | S | X | M
---+-----------+-----+----------+-----+-----+---+---+--
0  | ·         | s1  | ·        | s2  | ·   | 4 | 5 | 3
1  | ·         | ·   | ·        | s6  | ·   | · | · | 7
2  | r6        | ·   | s8 / r6  | ·   | ·   | · | · | ·
3  | s9        | ·   | ·        | ·   | ·   | · | · | ·
4  | ·         | ·   | ·        | ·   | acc | · | · | ·
5  | ·         | ·   | ·        | ·   | r1  | · | · | ·
6  | s10 / r6  | ·   | r6       | ·   | ·   | · | · | ·
7  | ·         | ·   | s11      | ·   | ·   | · | · | ·
8  | ·         | ·   | ·        | ·   | r4  | · | · | ·
9  | ·         | ·   | ·        | ·   | r2  | · | · | ·
10 | ·         | ·   | ·        | ·   | r5  | · | · | ·
11 | ·         | ·   | ·        | ·   | r3  | · | · | ·

競合:
  ACTION[2, 'c'] = s8 / r6
  ACTION[6, 'a'] = s10 / r6
この文法は SLR(1) ではありません。
```

!python lr11.py ex261016g3.y を実行すると以下のようになる.

開始記号：S

生成規則：

```
  [1] S -> X
  [2] X -> M 'a'
  [3] X -> 'b' M 'c'
  [4] X -> 'd' 'c'
  [5] X -> 'b' 'd' 'a'
  [6] M -> 'd'
```

正準 LR(1) 構文解析表：

ACTION列：sN = Nへ shift、rN = 規則Nで reduce、acc = accept

状態	'a'	'b'	'c'	'd'	#	S	X	M
0	·	s1	·	s2	·	4	5	3
1	·	·	·	s6	·	·	·	7
2	r6	·	s8	·	·	·	·	·
3	s9	·	·	·	·	·	·	·
4	·	·	·	·	acc	·	·	·
5	·	·	·	·	r1	·	·	·
6	s10	·	r6	·	·	·	·	·
7	·	·	s11	·	·	·	·	·
8	·	·	·	·	r4	·	·	·
9	·	·	·	·	r2	·	·	·
10	·	·	·	·	r5	·	·	·
11	·	·	·	·	r3	·	·	·

競合はありません。この文法は LR(1) です。